IFAL-Maceió · Curso de Sistemas de Informação

Linguagens de Programação — 2º Período

Atividade 5 - Problemas de Programação

Nível Médio e Avançado · Capítulos 5, 6 e 7

Python · Java · C

Sebesta, R. W. Conceitos de Linguagens de Programação. 9. ed. Bookman, 2011.

Leo Fernandes, PhD
Instituto Federal de Alagoas · IFAL-Maceió

Instruções Gerais

Formato da atividade:

- Os problemas são apresentados no estilo de juízes online (ex.: Codeforces, SPOJ, Beecrowd). Cada problema define com precisão o formato de entrada e saída — leia com atenção.
- A atividade é iniciada em sala de aula e concluída em casa. Os problemas 1 a 4 são previstos para sala (nível médio); os problemas 5 a 8 são para casa (nível avançado).
- **Entrada de dados:** lida da entrada padrão (teclado/stdin) — use scanf/printf em C, Scanner ou BufferedReader em Java, input() em Python. Não use leitura de arquivos.
- **Saída de dados:** impressa na saída padrão (tela/stdout) — use printf em C, System.out.println em Java, print() em Python. Cada linha de saída deve terminar com '\n'. Não imprima mensagens como 'Digite um número:'.
- **Formato estrito:** a saída deve corresponder EXATAMENTE aos casos de exemplo — espaços, maiúsculas e quebras de linha importam. Juízes online são sensíveis a espaços extras.
- **Linguagem:** resolva na linguagem indicada em cada problema. Em problemas com múltiplas linguagens, a solução em todas as linguagens é exigida.

Conceitos obrigatórios por problema:

- Cada problema exige construções específicas dos Caps. 5, 6 e 7. O uso dessas construções é obrigatório — soluções que as evitem não serão aceitas mesmo que produzam a saída correta.
- Os conceitos exigidos estão listados em cada enunciado na seção 'Exigências de construção'.

Nível Médio (Problemas 1–4)	Nível Avançado (Problemas 5–8)
Conceitos diretos. Solução em 20–40 min. Para sala de aula.	Combinação de conceitos. Solução em 40–90 min. Para casa.

Problemas de Nível Médio — Para Sala de Aula

Problema 1 Médio Caps. 5, 6 e 7 **Python · Java**

O Cofre das Variáveis

Uma instituição financeira armazena saldos de contas usando três tipos diferentes de dados: inteiro, ponto flutuante e ponto flutuante de dupla precisão. Dado um valor monetário lido como string no formato 'XXXXX.CC' (onde X são dígitos inteiros e C são centavos), o sistema precisa convertê-lo para cada tipo, realizar uma operação aritmética e reportar os resultados — incluindo as perdas de precisão geradas pelas conversões.

Entrada:

- Uma linha com um número real positivo no formato decimal com duas casas fracionárias (ex.: 1234.56). O valor estará sempre entre 0.01 e 99999.99.

Saída:

- Quatro linhas, nesta ordem:

Linha 1: o valor como inteiro (truncado, sem arredondamento) — ex.: 1234

Linha 2: o valor como float (precisão de 2 casas decimais) — ex.: 1234.56

Linha 3: o valor como double (precisão de 2 casas decimais) — ex.: 1234.56

Linha 4: a diferença absoluta entre float e double, com 6 casas decimais — ex.: 0.000000

Entrada	Saída esperada

`1234.56`	`1234`
	`1234.56`
	`1234.56`
	`0.000000`

`0.01`	`0`
	`0.01`
	`0.01`
	`0.000000`

`99999.99`	`99999`
	`99999.99`
	`99999.99`
	`0.000000`

Exigências de construção:

- Java: usar int, float e double; usar Scanner; usar cast (int) e (float); usar printf ou String.format.
- Python: usar int(), float() e conversão manual; usar input(); usar f-string com :.2f e :.6f.
- Cap. 6 — demonstrar coerção de estreitamento (float→int) e observar a perda de precisão.
- Cap. 7 — usar expressão de modo misto e cast explícito.

Problema 2 Médio Caps. 5 e 7 **Python · Java**

O Avaliador de Expressões

Em uma competição de lógica matemática, os participantes devem avaliar expressões aritméticas compostas sem o auxílio de calculadora. Seu programa deve avaliar uma expressão com os quatro operadores básicos e reportar o resultado, além de identificar se a expressão gerou divisão inteira ou real — expondo as regras de precedência e associatividade da linguagem usada.

Entrada:

- Três inteiros A, B e C ($-1000 \leq A, B, C \leq 1000$, $C \neq 0$), cada um em uma linha separada.

Saída:

- Linha 1: resultado de A + B * C (demonstra precedência: * antes de +)
- Linha 2: resultado de A * B + B * C (associatividade à esquerda)
- Linha 3: resultado de A / B / C usando divisão INTEIRA (escreva 'INTEIRO: X')
- Linha 4: resultado de A / B / C usando divisão REAL com 4 casas decimais (escreva 'REAL: X.XXXX')
- Linha 5: 'POSITIVO', 'NEGATIVO' ou 'ZERO' conforme o sinal de A + B * C

Atenção: B pode ser 0. Se B = 0, as linhas 3 e 4 devem escrever 'INDEFINIDO'.

Entrada	Saída esperada
3 4 2	11 14 INTEIRO: 0 REAL: 0.3750 POSITIVO
10 0 5	10 0 INDEFINIDO INDEFINIDO POSITIVO
-6 3 -2	0 -24 INTEIRO: 1 REAL: 1.0000 ZERO

Exigências de construção:

- Java: usar / para divisão inteira entre int; (double)A/B/C para divisão real; demonstrar que A/B/C é avaliado da esquerda para a direita (associatividade).
- Python: usar // para divisão inteira; / para real; usar abs() para verificar zero.
- Cap. 7 — precedência (* antes de +), associatividade à esquerda da divisão e expressões booleanas na verificação do sinal.
- Cap. 5 — usar variável auxiliar para armazenar resultado intermediário (variável dinâmica da pilha).

Problema 3 Médio Caps. 5 e 6 C

O Mapa de Memória

Em sistemas embarcados, os engenheiros precisam gerenciar manualmente a memória. Seu programa em C simula um registro de sensores usando estrutura (struct), acessa campos via ponteiro e reporta os endereços de memória e os valores armazenados — expondo como variáveis são organizadas na memória.

Entrada:

- Três linhas: um inteiro (ID do sensor), um float (temperatura) e um char (status: 'A', 'I' ou 'E').

Saída:

- Linha 1: ID do sensor
- Linha 2: temperatura com 2 casas decimais
- Linha 3: status
- Linha 4: 'SIM' se temperatura > 50.0 E status == 'A'; 'NAO' caso contrário
- Linha 5: resultado de (id * 2 + (int)temperatura) usando operadores compostos — escreva 'COMPOSTO: X'

Entrada	Saída esperada
42 67.3 A	42 67.30 A SIM COMPOSTO: 151
7 23.1 I	7 23.10 I NAO COMPOSTO: 37
100 50.0 A	100 50.00 A NAO COMPOSTO: 250

Exigências de construção:

- C obrigatório — usar struct com pelo menos 3 campos (int, float, char).
- Declarar um PONTEIRO para a struct e acessar todos os campos via operador -> (não via variável direta).
- Usar cast explícito (int) na conversão de float para int na expressão da Linha 5.
- Usar operador de atribuição composto (+=, *=, ou similar) em pelo menos uma etapa do cálculo.
- Cap. 5 — a struct e o ponteiro são variáveis dinâmicas da pilha.
- Cap. 7 — expressar a condição da Linha 4 com && (avaliação em curto-circuito).

Problema 4 Médio Caps. 6 e 7 **Python · Java**

O Classificador de Tipos

Um sistema de diagnóstico recebe N valores e precisa classificá-los dinamicamente: determinar se cada um é inteiro, real ou booleano em Python — e comparar com o comportamento estático de Java, onde o tipo é fixo em compilação. Isso expõe a diferença entre tipagem estática e dinâmica.

Entrada:

- Primeira linha: inteiro N (1 ≤ N ≤ 10) — número de valores.
- As N linhas seguintes: cada uma com uma string que representa um valor — pode ser um inteiro, um decimal com ponto, ou 'True'/'False'.

Saída (Python):

- Para cada valor: o tipo detectado ('int', 'float' ou 'bool') e o valor convertido. Se o valor for bool, imprimir também o inteiro equivalente.
- Após os N valores: a SOMA de todos os valores numéricos (int + float) com 2 casas decimais.

Nota:

- Teste 'True'/'False' ANTES de testar int/float — em Python, bool é subclasse de int e isinstance(True, int) retorna True.

Entrada	Saída esperada
4 42 3.14 True 7	int: 42 float: 3.14 bool: True (int=1) int: 7 SOMA: 52.14

Exigências de construção:

- Python — usar isinstance() para verificar tipos na ordem: bool, int, float.
- Python — usar try/except para tentar conversões: primeiro bool (verificar 'True'/'False'), depois int(), depois float().
- Python — usar uma lista para acumular os valores; calcular a soma com sum() ou loop com +=.
- Java — implementar o equivalente com tipos declarados (double para todos); comentar no código onde Java seria incapaz de determinar o tipo dinamicamente.
- Cap. 6 — diferença entre tipagem dinâmica (Python) e estática (Java); coerção implícita bool→int em Python.
- Cap. 7 — usar operador de atribuição acumulada (+=) no cálculo da soma.

Problemas de Nível Avançado — Para Casa

Problema 5 Avançado Caps. 5, 6 e 7 C

O Gerenciador de Pilha Dinâmica

Em sistemas de baixo nível, pilhas são implementadas manualmente com arrays e ponteiros. Seu programa em C implementa uma pilha de inteiros usando alocação dinâmica no heap (malloc/free) e manipulação direta de ponteiros. Ele deve processar uma sequência de operações PUSH, POP e TOP, reportando o estado após cada operação.

Entrada:

- Primeira linha: inteiro N (1 ≤ N ≤ 20) — número de operações.
- As N linhas seguintes: cada uma com uma operação no formato 'PUSH X' (X inteiro) ou 'POP' ou 'TOP'.

Saída:

- Para PUSH X: imprimir 'PUSH: X'
- Para POP: imprimir 'POP: X' com o valor removido, ou 'ERRO: pilha vazia'
- Para TOP: imprimir 'TOP: X' com o valor do topo sem remover, ou 'ERRO: pilha vazia'
- Ao final: imprimir 'TAMANHO: X' com o número de elementos restantes na pilha

Entrada	Saída esperada
6 PUSH 10 PUSH 20 TOP POP POP POP	PUSH: 10 PUSH: 20 TOP: 20 POP: 20 POP: 10 ERRO: pilha vazia TAMANHO: 0

Exigências de construção:

- C obrigatório — usar malloc() para alocar o array de inteiros no heap (variável dinâmica do monte explícita, Sebesta Cap. 5).
- Usar free() ao final do programa para liberar a memória alocada.
- Toda manipulação do array deve ser feita via PONTEIRO (int *pilha); proibido usar indexação direta pilha[i] na implementação das operações — usar aritmética de ponteiros *(pilha + i).
- Usar um ponteiro separado int *topo que aponta para o elemento do topo.
- Cap. 5 — contrastar: a variável ponteiro (pilha, topo) é dinâmica da pilha; o array alocado com malloc é dinâmico do monte explícito.
- Cap. 7 — usar pré-incremento (++topo_idx) e pós-decremento (topo_idx--) nas operações PUSH e POP.

Problema 6 Avançado Caps. 6 e 7 Java · Python

O Analisador de Sequências

Uma plataforma de análise financeira recebe sequências de valores e precisa calcular estatísticas, detectar anomalias de tipo e gerar um relatório formatado. O sistema deve lidar com arrays de tamanho variável, conversões entre tipos e expressões complexas — demonstrando as diferenças entre Java (tipagem estática, arrays fixos) e Python (tipagem dinâmica, listas flexíveis).

Entrada:

- Primeira linha: inteiro N (1 ≤ N ≤ 100).
- Segunda linha: N números reais separados por espaço.

Saída:

- Linha 1: 'MIN: X.XX' — menor valor
- Linha 2: 'MAX: X.XX' — maior valor
- Linha 3: 'MEDIA: X.XX' — média aritmética
- Linha 4: 'VARIANCIA: X.XXXX' — variância amostral (dividir por N-1; se N=1: escrever 'INDEFINIDA')
- Linha 5: 'INTEIROS: K' — quantos valores são inteiros exatos (parte fracionária = 0)
- Linha 6: 'ACIMA_MEDIA: K' — quantos valores estão estritamente acima da média

Entrada	Saída esperada
5 3.0 1.5 4.0 1.5 5.0	MIN: 1.50 MAX: 5.00 MEDIA: 3.00 VARIANCIA: 2.5000 INTEIROS: 3 ACIMA_MEDIA: 2
1 7.25	MIN: 7.25 MAX: 7.25 MEDIA: 7.25 INDEFINIDA INTEIROS: 0 ACIMA_MEDIA: 0

Exigências de construção:

- Java — usar array double[] de tamanho N; ler com Scanner; usar laços for com índice explícito; usar Math.min(), Math.max().
- Java — calcular variância usando expressão com Math.pow() ou operador * e atribuição composta +=; fazer cast (int) para verificar se um valor é inteiro exato.
- Python — usar list comprehension para ler os valores: [float(x) for x in input().split()]; usar sum(), min(), max() nativos.
- Python — calcular variância com expressão de modo misto: soma de (x - media)**2 usando generator expression.
- Cap. 6 — verificar se valor é inteiro exato: em Java, v == (int)v (coerção); em Python, v == int(v).

- Cap. 7 — usar operador ternário em Python (valor if cond else valor) e expressão condicional em Java.

Problema 7 Avançado Caps. 5, 6 e 7 C · Python

O Detetive de Escopo

Uma ferramenta de auditoria de código precisa simular o comportamento de variáveis em diferentes escopos e reportar os valores acessíveis em cada ponto de execução. Em C, o programa usa blocos aninhados com variáveis de mesmo nome (sombreamento); em Python, demonstra escopo léxico e o comportamento de global/nonlocal. O programa recebe uma sequência de operações e reporta o valor da variável x em cada ponto.

Entrada:

- Primeira linha: inteiro N (1 ≤ N ≤ 15).
- As N linhas seguintes: operações no formato:

'SET_GLOBAL X' — define x global como X

'SET_LOCAL X' — define x local (escopo interno) como X

'READ_GLOBAL' — imprime o valor do x global

'READ_LOCAL' — imprime o valor do x local (se não houver local, usa o global)

'RESET_LOCAL' — destrói o x local (volta a usar o global)

Saída:

- Para cada READ: imprimir 'GLOBAL: X' ou 'LOCAL: X' indicando qual variável está sendo lida
- Ao final: imprimir 'HISTORICO: G=X L=Y' com os valores atuais de global e local (ou L=NENHUM se não houver local)

Entrada	Saída esperada
7 SET_GLOBAL 10 SET_LOCAL 20 READ_LOCAL READ_GLOBAL RESET_LOCAL READ_LOCAL SET_GLOBAL 99	LOCAL: 20 GLOBAL: 10 LOCAL: 10 HISTORICO: G=99 L=NENHUM

Exigências de construção:

- C — implementar com blocos aninhados { } reais; a variável local deve ser declarada dentro de um bloco aninhado; usar variável global com especificador static (persistente entre chamadas de simulação).
- C — usar ponteiro para int (*ptr_global, *ptr_local) para acessar as variáveis; toda leitura deve ser via ponteiro.

- Python — implementar com dicionário para simular os escopos; usar a palavra-chave global explicitamente; demonstrar que sem global a função não pode modificar a variável externa.
- Cap. 5 — demonstrar explicitamente: (1) sombreamento (variável local oculta a global); (2) tempo de vida diferente (local é destruída ao sair do bloco); (3) ambiente de referenciamento.
- Cap. 6 — garantir que os valores são armazenados como int; não usar strings para guardar números.
- Cap. 7 — usar operador ternário ou expressão condicional para selecionar qual valor imprimir (local ou global).

Problema 8 Avançado Caps. 5, 6 e 7 **Python · Java · C**

O Codificador de Dados

Um sistema de transmissão de dados precisa empacotar informações de tipos heterogêneos em uma representação compacta, aplicar operações bit a bit e de ponto flutuante, e verificar a integridade do pacote. Este problema combina arrays, ponteiros (C), tipos de referência (Java), manipulação de strings e expressões complexas nas três linguagens.

Entrada:

- Linha 1: inteiro K ($1 \leq K \leq 8$) — número de valores no pacote.
- Linha 2: K inteiros separados por espaço ($0 \leq vi \leq 255$).
- Linha 3: um número real R ($0.0 < R \leq 1.0$) — fator de escala.

Saída:

- Linha 1: 'SOMA_BRUTA: X' — soma dos K inteiros
- Linha 2: 'SOMA_ESCALADA: X.XX' — soma * R
- Linha 3: 'XOR_CHECK: X' — XOR acumulado de todos os K valores (checksum)
- Linha 4: 'MEDIA_INT: X' — média truncada para inteiro
- Linha 5: 'ACIMA_MEDIA: v1 v2 ...' — valores acima da média inteira, em ordem, separados por espaço; se nenhum: 'NENHUM'
- Linha 6: 'STATUS: OK' se XOR_CHECK == 0; 'STATUS: ERRO X' onde X é o valor de XOR_CHECK

Entrada	Saída esperada
4 10 20 30 40 0.5	SOMA_BRUTA: 100 SOMA_ESCALADA: 50.00 XOR_CHECK: 20 MEDIA_INT: 25 ACIMA_MEDIA: 30 40 STATUS: ERRO 20
3 5 10 15 1.0	SOMA_BRUTA: 30 SOMA_ESCALADA: 30.00 XOR_CHECK: 0 MEDIA_INT: 10 ACIMA_MEDIA: 15 STATUS: OK

Exigências de construção:

- C — usar array int v[8]; ler com scanf em loop; calcular XOR com operador ^ em loop com atribuição composta (xor ^= v[i]); usar ponteiro int *p = v para percorrer o array.
- Java — usar int[] com tamanho K; usar operador ^ para XOR; usar (int) para truncar a divisão na média; usar StringBuilder para construir a linha ACIMA_MEDIA.
- Python — usar list comprehension para leitura; usar operador ^ em reduce (from functools import reduce) ou em loop com ^=; usar // para divisão inteira na média.
- Cap. 6 — operador XOR (^) aplicado a int; coerção de estreitamento na média (resultado da divisão truncado para int); modo misto soma_bruta * R onde int*float = float.
- Cap. 7 — operadores de atribuição composta (^=, +=); expressão condicional para o STATUS; ordem de avaliação na linha ACIMA_MEDIA.
- Cap. 5 — em C, contrastar v (array estático na pilha) com o array alocado dinamicamente do Problema 5.

Referência Rápida — Entrada e Saída

Leitura da entrada padrão

C	`scanf("%d %f %c", &n, &f, &c);  // int, float, char`
Java	`Scanner sc = new Scanner(System.in);  int n = sc.nextInt();  double d = sc.nextDouble();`
Python	`n = int(input())  # ou:  a, b = map(int, input().split())  # ou:  vals = [float(x) for x in input().split()]`

Escrita na saída padrão

C	`printf("%d\n", n);  printf("%.2f\n", f);  printf("MIN: %.2f\n", min);`
Java	`System.out.println(n);  System.out.printf("%.2f%n", d);  System.out.printf("MIN: %.2f%n", min);`
Python	`print(n)  print(f"{d:.2f}")  print(f"MIN: {min_val:.2f}")`

Regras de formatação para juízes online

- **Nunca** imprima mensagens de prompt como 'Digite um número:' ou 'Informe N:' — o juiz compara sua saída com a esperada exatamente.
- Cada linha de saída deve terminar com exatamente uma quebra de linha (\n). Não imprima linhas em branco extras.
- Números reais: use sempre o número de casas decimais indicado no enunciado. Ex.: se o enunciado pede 2 casas, 3.0 deve ser impresso como '3.00', não '3'.
- Strings de texto (ex.: 'SIM', 'ERRO', 'STATUS: OK'): maiúsculas e minúsculas importam. Copie o formato exato do enunciado.
- Múltiplos valores na mesma linha: separados por espaço simples, sem espaço no final da linha.